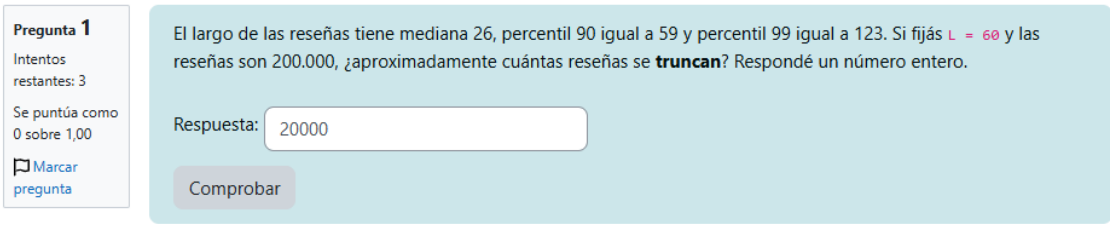
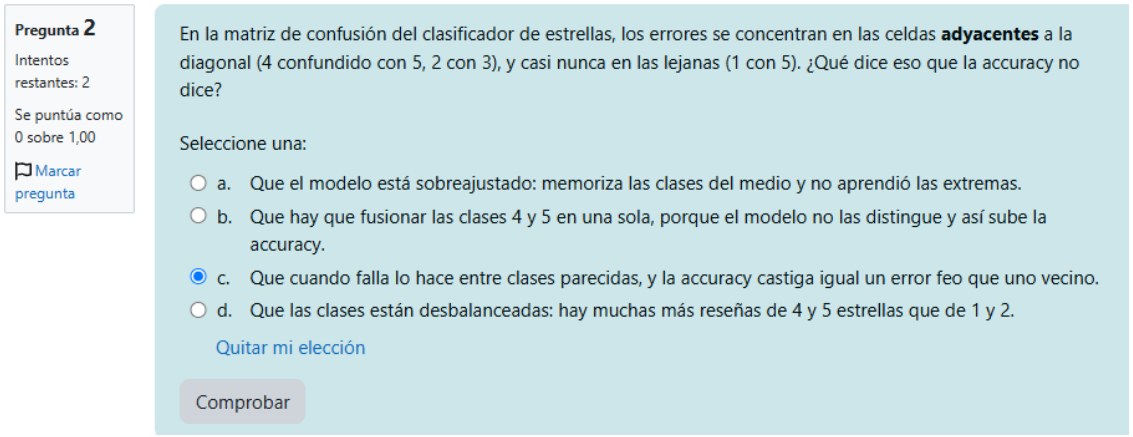
Primer Test



El percentil 90 vale 59: el 10 % de las reseñas es más largo que 59, y con L = 60 casi todas ésas se truncan. 10 % de 200.000 ≈ 20.000 (el notebook mide 10 % exacto para L = 60).

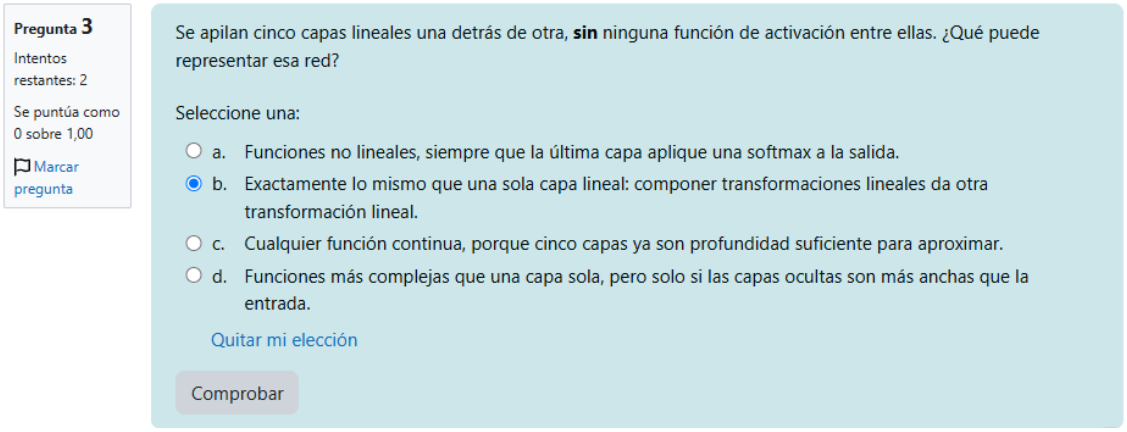
Resolución. El percentil 90 es 59 palabras, así que el 10 % de las reseñas supera ese largo. Con L = 60 se truncan ≈ 10 % × 200.000 = **20.000**. Subir L a 123 (percentil 99) bajaría el truncado al 1 % al doble de cómputo; bajarlo a la mediana truncaría la mitad.

La respuesta correcta es: 20000



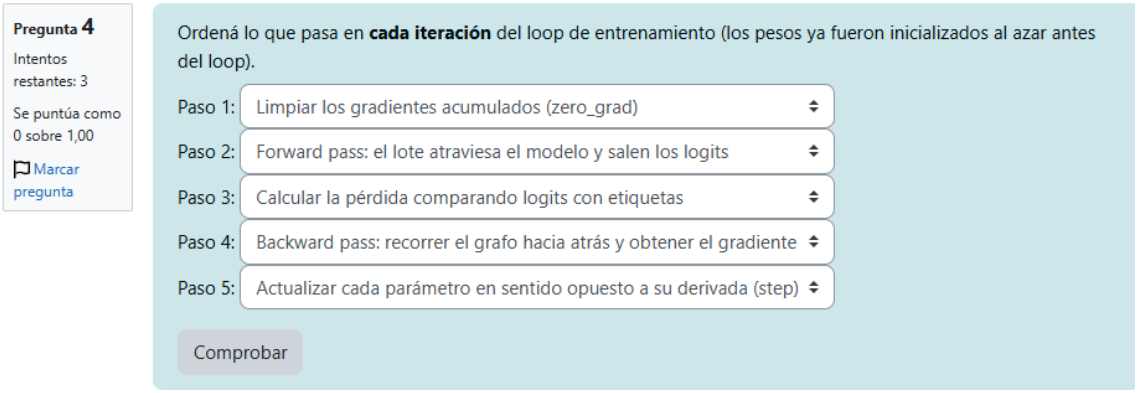
Sí. La diferencia entre 4 y 5 estrellas es ambigua incluso para una persona. Un número resumen no distingue un error de una estrella de uno de cuatro; la matriz sí.
Resolución. Todos los aciertos caen en la diagonal; lo interesante es el entorno. Errores concentrados en estrellas adyacentes significan que el modelo confunde clases genuinamente parecidas y casi nunca se equivoca feo. La accuracy «cruda» castiga igual confundir 4 con 5 que 1 con 5; la matriz muestra con qué confunde cada cosa.

La respuesta correcta es: Que cuando falla lo hace entre clases parecidas, y la accuracy castiga igual un error feo que uno vecino.



Sí. En el notebook se comprueba con números: dos capas sin activación son idénticas a una capa con la matriz W2 @ W1. Toda la profundidad no compra nada.
Resolución. W2 (W1 x + b1) + b2 = (W2 W1) x + (W2 b1 + b2): una sola capa. Con una ReLU en el medio ya no existe ninguna matriz que reproduzca el cálculo, y con suficientes unidades el MLP se vuelve un aproximador universal. Cada quiebre de la función de salida es una ReLU activándose o apagándose.

La respuesta correcta es: Exactamente lo mismo que una sola capa lineal: componer transformaciones lineales da otra transformación lineal.



Resolución. zero_grad → forward → pérdida → backward → step. Las tres líneas de PyTorch, siempre en ese orden: zero_grad(), backward(), step(). Limpiar al principio (o justo antes de backward) evita que el gradiente de este lote se sume al del anterior. No hay gradiente antes de la pérdida, y no hay pérdida antes del forward.

Pregunta 5

Intentos restantes: 2

Se puntúa como 0 sobre 1,00

Marcar pregunta

La teoría recomienda normalizar las variables de entrada de una red (llevarlas a media 0 y desvío 1, o al rango 0–1). La entrada de este modelo es un tensor de **índices de vocabulario**, números enteros entre 0 y 23.000 donde cada uno identifica una palabra. ¿Se normalizan?

Seleccione una:

- a. No: los índices son casilleros de una tabla, no cantidades; el rol de la normalización lo cumple el embedding.
- b. Sí: dividir por el tamaño del vocabulario deja los índices entre 0 y 1 y evita que las palabras raras, de índice alto, dominen.
- c. No hace falta porque `nn.Embedding` ya normaliza internamente cada fila de la tabla a norma 1 al inicializarla.
- d. Sí, pero con media 0 y desvío 1, porque los índices tienen una distribución muy sesgada hacia las palabras frecuentes.

Quitar mi elección

Comprobar

Exacto. El token 15.000 no es «más grande» que el 3. La capa de embedding convierte cada índice en un vector de floats con una escala razonable, aprendida.
Resolución. Normalizar supone que los números son cantidades comparables. Los índices son etiquetas: cualquier operación aritmética sobre ellos es absurda. La transformación que en este problema cumple el rol de la normalización es la **capa de embedding**; en arquitecturas más grandes reaparece la normalización, pero sobre activaciones internas (*layer normalization*).

La respuesta correcta es: No: los índices son casilleros de una tabla, no cantidades; el rol de la normalización lo cumple el embedding.

Pregunta 6

Intentos restantes: 2

Se puntúa como 0 sobre 1,00

Marcar pregunta

Se arma el vocabulario con las palabras del corpus de **entrenamiento** que aparecen al menos unas cuantas veces; el resto va a parar a `<unk>`. Ese vocabulario tiene **cobertura 99 %** y, aplicado a textos de *test* que nunca se usaron para construirlo, una **tasa de `<unk>` de 1,2 %**. ¿Qué mide cada uno de esos dos números?

Seleccione una:

- a. La cobertura mira el corpus con que se construyó el vocabulario; la tasa de `<unk>`, cualquier texto nuevo. Por eso pueden diferir.
- b. Son la misma medida desde dos lados: cobertura = 1 − tasa de `<unk>`, y la diferencia es error de redondeo.
- c. La cobertura se mide sobre test y la tasa de `<unk>` sobre entrenamiento, para detectar sobreajuste del vocabulario.
- d. La cobertura cuenta formas distintas del vocabulario; la tasa de `<unk>` cuenta apariciones.

Quitar mi elección

Correcto. Cobertura = fracción de apariciones del corpus de entrenamiento que quedan dentro del vocabulario. Tasa de <unk> = fracción de tokens de un texto cualquiera (acá, test) que caen en <unk>, incluidos los que nunca aparecieron en entrenamiento.
Resolución. cobertura se calcula con el contador del corpus de entrenamiento; tasa_unk(textos) recibe textos cualesquiera. Con freq_min=1 la cobertura es 100 % y la tasa de <unk> en test es 0,23 %: ese 0,23 % es el piso irreducible, palabras que simplemente no estaban en entrenamiento.

La respuesta correcta es: La cobertura mira el corpus con que se construyó el vocabulario; la tasa de <unk>, cualquier texto nuevo. Por eso pueden diferir.

Pregunta 7

Intentos restantes: 2

Se puntúa como 0 sobre 1,00

Marcar pregunta

Con los ejemplos en filas (`X` de forma `(B, entradas)`) y la matriz de pesos `W` de forma `(salidas, entradas)`, ¿cómo se calcula la capa entera de una sola vez?

Seleccione una:

- a. `W @ X + b`: los pesos van primero porque cada fila de `W` es un perceptrón.
- b. `X @ W.T + b`: la transposición hace coincidir las dimensiones internas, y `b` se estira por broadcasting a cada fila.
- c. `X * W + b`: el producto elemento a elemento multiplica cada entrada por su peso y el broadcasting suma.
- d. Un `for` sobre las B filas, aplicando `W @ x + b` a cada una; PyTorch lo paraleliza solo.

Quitar mi elección

Comprobar

Sí. (B, entradas) @ (entradas, salidas) = (B, salidas), y el sesgo (salidas,) se suma a cada fila del lote.

Resolución. Con ejemplos en filas, z = X @ W.T + b. La regla de las dimensiones internas (la segunda de la primera matriz = la primera de la segunda) es la fuente de la mitad de los errores del cuatrimestre: cuando algo falle, imprimí las formas. Y el sesgo se suma por broadcasting a cada fila.

La respuesta correcta es: X @ W.T + b: la transposición hace coincidir las dimensiones internas, y b se estira por broadcasting a cada fila.

Pregunta 8

Intentos restantes: 3

Se puntúa como 0 sobre 1,00

Marcar pregunta

El corpus de reseñas viene **ordenado por etiqueta**: primero todas las de 1 estrella, después las de 2, y así hasta las de 5. Con ese corpus, ¿por qué `shuffle=True` en el DataLoader de entrenamiento no es un detalle cosmético?

Seleccione una:

- a. Porque en validación y prueba también hay que barajar, y el DataLoader lo hace de forma consistente en los tres.
- b. Porque barajar hace estocástico el descenso por mini-lotes: la red no ve siempre las mismas secuencias de ejemplos.
- c. Porque sin barajar el DataLoader produce siempre el mismo primer lote y el modelo solo aprende de él.
- d. Porque barajar reduce el tamaño efectivo del lote y hace las actualizaciones más frecuentes.

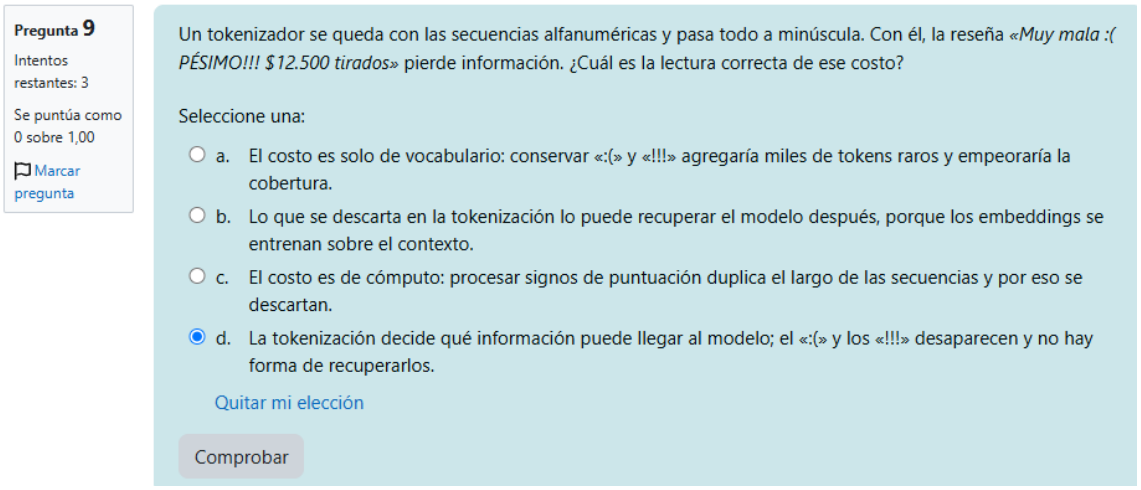
Quitar mi elección

Comprobar

Sí. Y en este corpus es más grave: viene ordenado por clase, así que sin barajar cada época mostraría las cinco clases una después de la otra, en bloque.

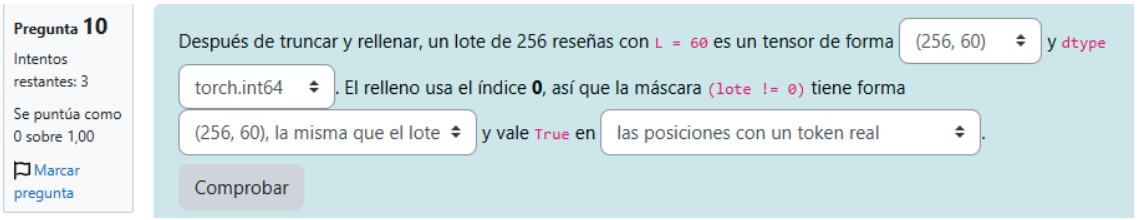
Resolución. Sin barajar, la red ve los ejemplos siempre en el mismo orden (y acá, ordenados por clase): las actualizaciones desarrollan patrones que se cancelan y el entrenamiento se degrada. «Estocástico» en SGD alude justamente a la selección aleatoria. Validación y prueba no se barajan: el orden no afecta el resultado y la reproducibilidad importa.

La respuesta correcta es: Porque barajar hace estocástico el descenso por mini-lotes: la red no ve siempre las mismas secuencias de ejemplos.



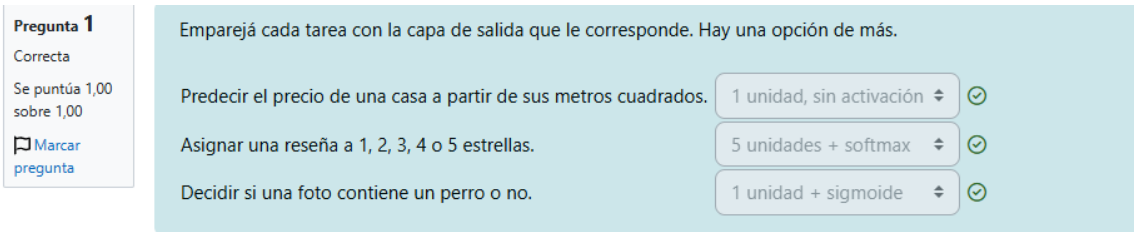
Exacto. Es la idea central de la sección: no es preprocesamiento aburrido, es dónde se decide qué llega. Lo que se tira acá el modelo no lo ve nunca.
Resolución. El tokenizador es la primera puerta: lo que no pasa por ahí no llega al modelo. tok_simple hace desaparecer «:(», los «!!!» y parte «$12.500» en «12» y «500». Medido en el notebook: conservar puntuación cuesta una decena de tokens de vocabulario y un 11 % de largo; conservar mayúsculas cuesta 20 % de vocabulario y *empeora* las métricas.

La respuesta correcta es: La tokenización decide qué información puede llegar al modelo; el «:(» y los «!!!» desaparecen y no hay forma de recuperarlos.



Resolución. (B, L) = (256, 60), enteros de 64 bits porque son **índices** (el 42 no es «más grande» que el 7). La máscara es la comparación elemento a elemento con pad_id = 0, así que tiene la misma forma y es True en los tokens reales. Esa máscara vuelve más adelante, para promediar sin contar el relleno.

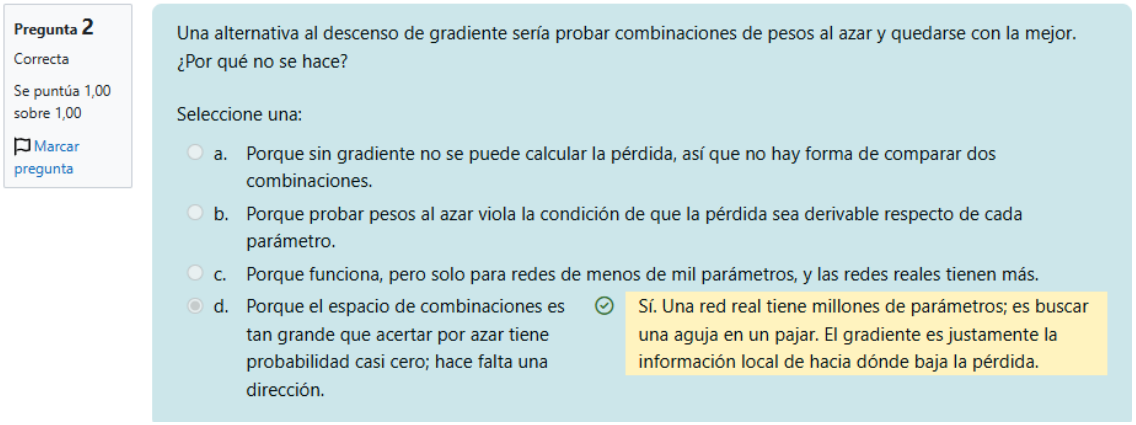
Segundo Test



Bien: regresión → una salida cruda; binaria → una sigmoide; multiclase → una unidad por clase con softmax.

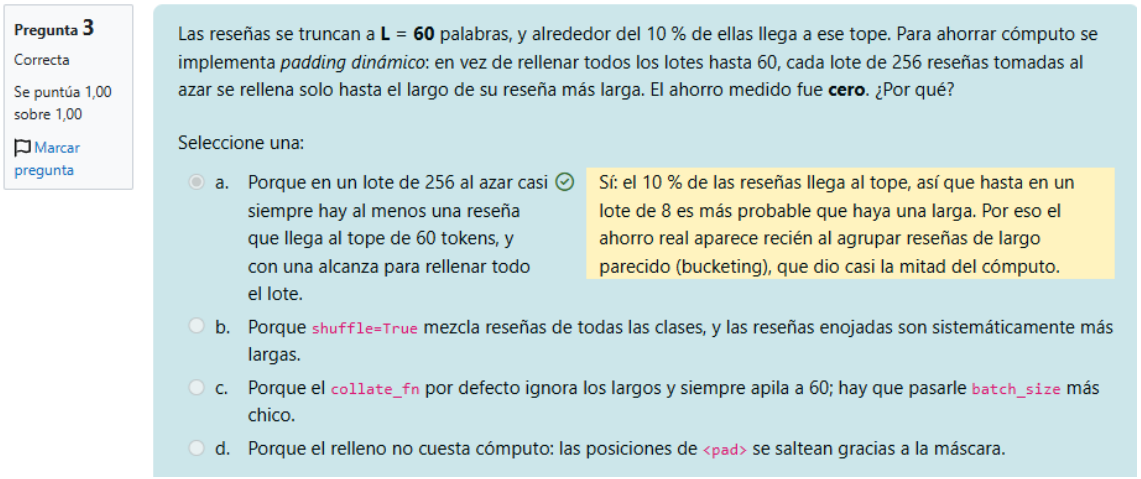
Resolución. La capa de salida la dicta la tarea: **regresión** → 1 unidad y el valor es la predicción; **binaria** → 1 unidad + sigmoide, probabilidad de la clase positiva; **multiclase** → una unidad por clase + softmax. «2 unidades + sigmoide» sobra: dos sigmoides independientes no forman una distribución.

La respuesta correcta es: Predecir el precio de una casa a partir de sus metros cuadrados. → 1 unidad, sin activación, Asignar una reseña a 1, 2, 3, 4 o 5 estrellas. → 5 unidades + softmax, Decidir si una foto contiene un perro o no. → 1 unidad + sigmoide



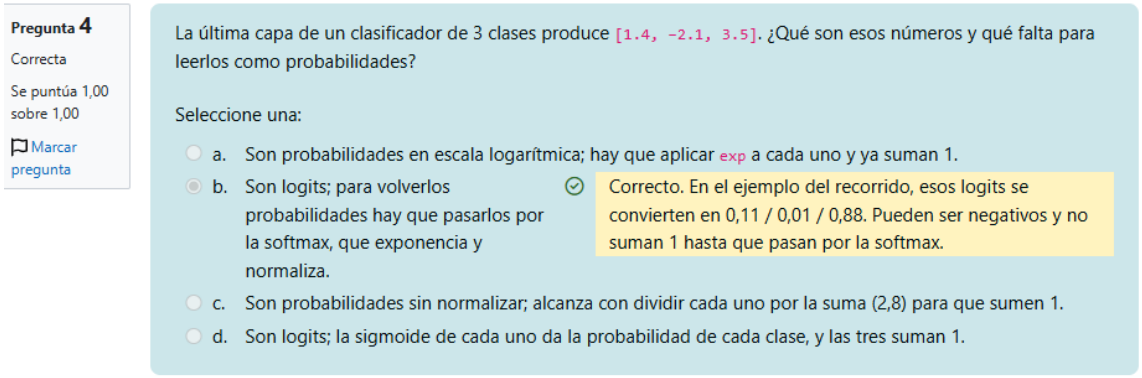
Resolución. El espacio de parámetros es enorme y la probabilidad de acertar por azar es nula en la práctica. El descenso de gradiente reemplaza la suerte por información local: la pendiente en el punto actual dice hacia qué lado baja la pérdida, y repetir ese paso es todo el algoritmo.

La respuesta correcta es: Porque el espacio de combinaciones es tan grande que acertar por azar tiene probabilidad casi cero; hace falta una dirección



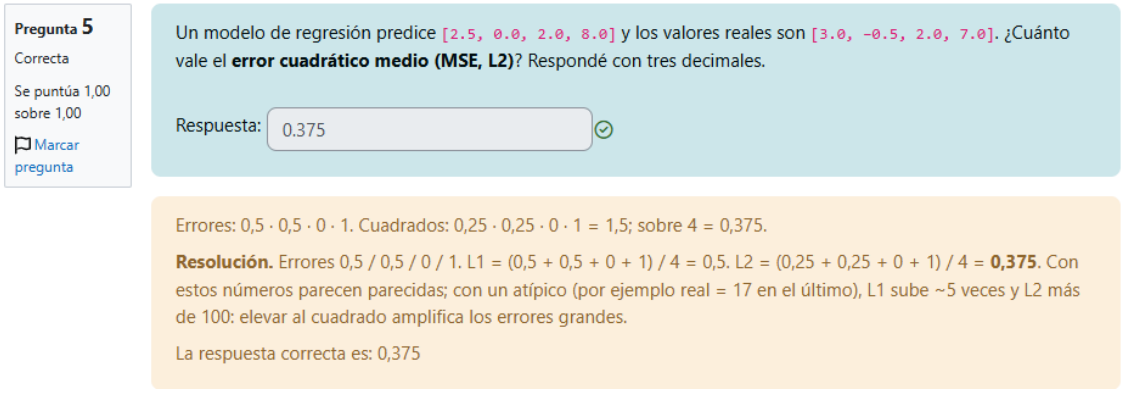
Resolución. El largo máximo de un lote de 256 al azar es casi siempre el máximo posible: alcanza con una reseña larga. La solución es agrupar por largo (*bucketing*: ordenar, cortar lotes, barajar el orden de los lotes con batch_sampler), que ahorró casi la mitad del cómputo a costa de que los lotes dejen de ser muestras aleatorias.

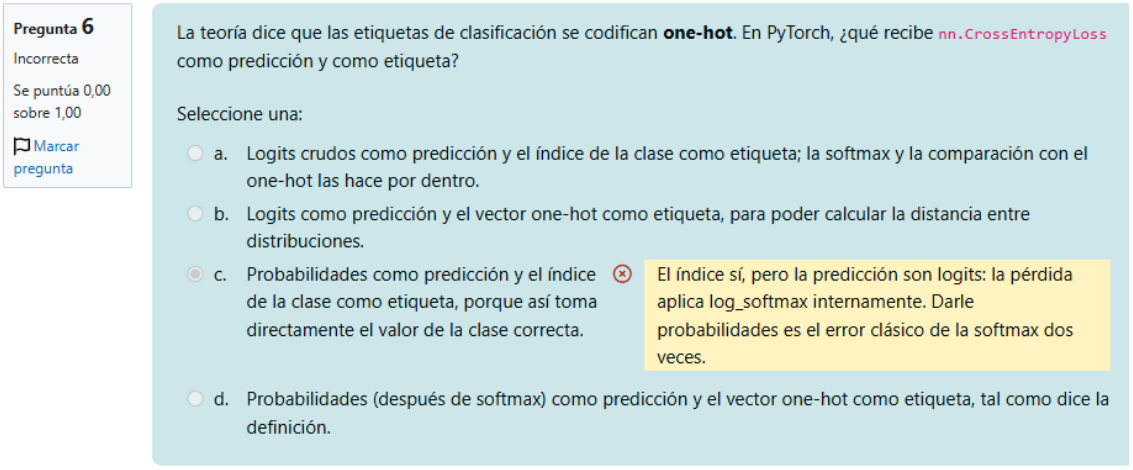
La respuesta correcta es: Porque en un lote de 256 al azar casi siempre hay al menos una reseña que llega al tope de 60 tokens, y con una alcanza para rellenar todo el lote.



Resolución. Los valores crudos de la última capa son **logits**: pueden ser negativos y no suman 1. $\text{softmax}(z)_i = \exp(z_i) / \Sigma \exp(z_j)$ los vuelve una distribución. Por convención el modelo devuelve logits y la softmax se aplica solo para leer probabilidades (la pérdida la aplica por dentro).

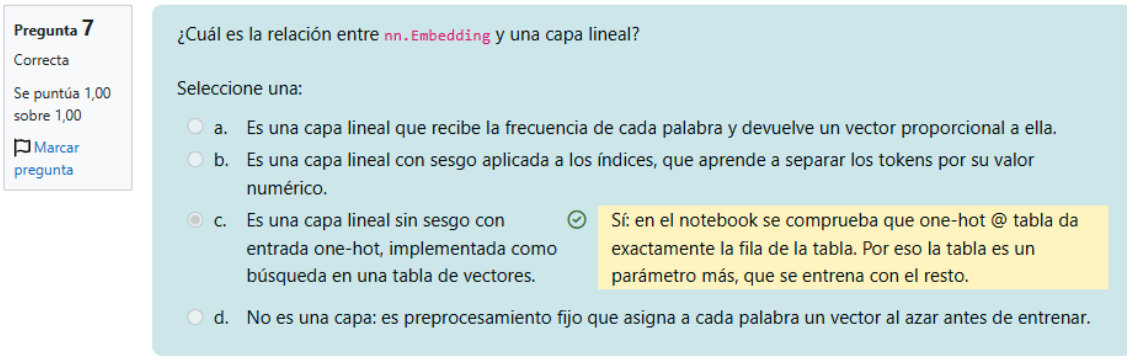
La respuesta correcta es: Son logits; para volverlos probabilidades hay que pasarlos por la softmax, que exponencia y normaliza.





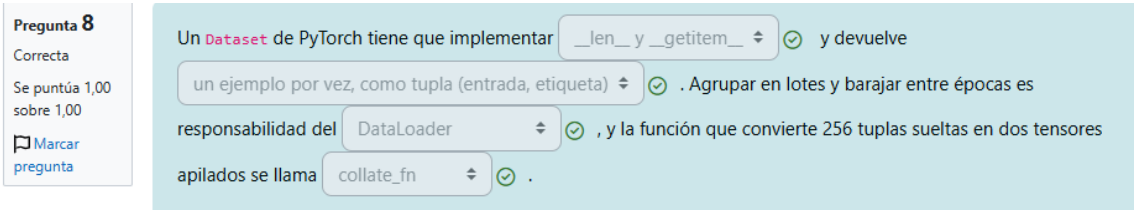
Resolución. Predicción: **logits** (N, clases). Etiqueta: **índice** (N,). Internamente: log_softmax y tomar el valor de la clase correcta con signo cambiado, que es la entropía cruzada contra el one-hot, de forma más estable y sin el tensor de ceros.

La respuesta correcta es: Logits crudos como predicción y el índice de la clase como etiqueta; la softmax y la comparación con el one-hot las hace por dentro.

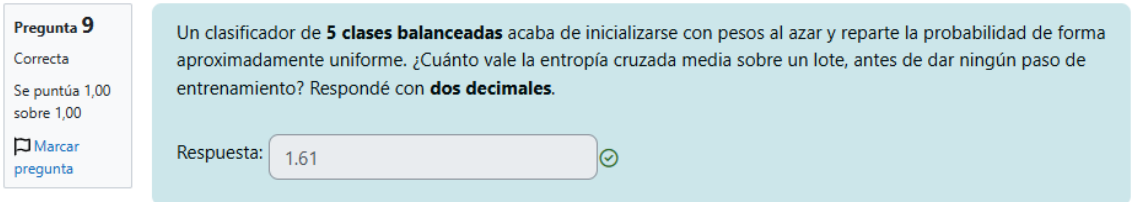


Resolución. nn.Embedding(V, E) es una tabla (V, E). Buscar la fila *i* es lo mismo que multiplicar el one-hot de *i* por la tabla; la búsqueda solo evita un producto absurdo con un vector de ceros. Consecuencias: es un parámetro más, y es la primera capa de cualquier modelo de lenguaje.

La respuesta correcta es: Es una capa lineal sin sesgo con entrada one-hot, implementada como búsqueda en una tabla de vectores.



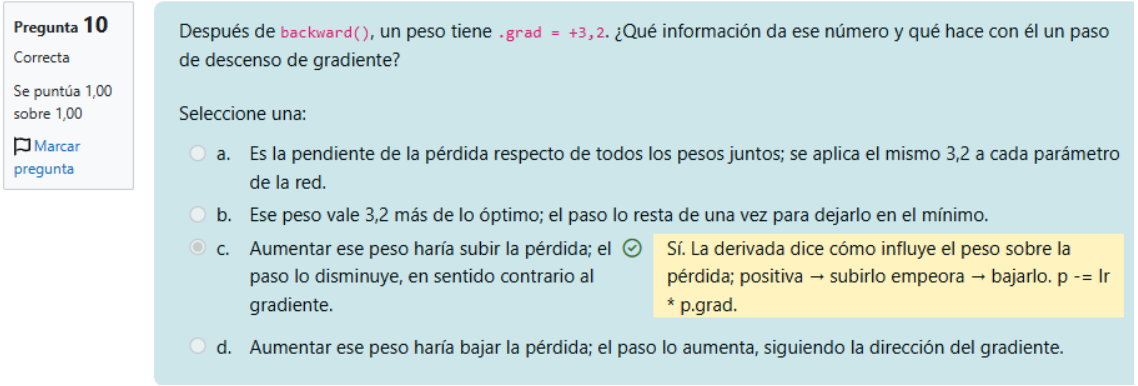
Resolución. Dataset: __len__ y __getitem__, un ejemplo por vez. DataLoader: lotes, barajado, paralelismo. collate_fn: convierte la lista de tuplas en tensores apilados (por defecto con torch.stack). batch_sampler elige qué índices van juntos: es lo que permite el *bucketing*.



ln(5) ≈ 1,609. Es la verificación que conviene hacer siempre: si la pérdida inicial no arranca cerca de ln(número de clases), hay un bug en la inicialización, en las etiquetas o en las formas.

Resolución. Uniforme sobre 5 clases → p(correcta) = 1/5 para cualquier ejemplo. Pérdida = −ln(1/5) = ln(5) ≈ **1,61**. En el notebook la celda da 1,60–1,61 según el lote, porque «aproximadamente uniforme» no es exactamente uniforme.

La respuesta correcta es: 1,61



Resolución. ∂pérdida/∂w = +3,2 significa que un aumento mínimo de w sube la pérdida. Regla: mover cada parámetro en sentido **opuesto** a su derivada, escalado por la tasa de aprendizaje: p -= lr * p.grad. El gradiente entero apunta hacia donde la pérdida crece más rápido; por eso el algoritmo se llama *descenso*.

La respuesta correcta es: Aumentar ese peso haría subir la pérdida; el paso lo disminuye, en sentido contrario al gradiente.

Pregunta 1
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un clasificador de texto con la misma arquitectura que el de reseñas, pero entrenado para predecir una de **10** categorías, recibe índices `(B, L)`. Después de `nn.Embedding(V, 64)` el tensor es (B, L, 64) ; después del promedio con máscara es (B, 64) ; y a la salida de la última `nn.Linear` es (B, 10) .

Resolución. (B, L) → embedding → (B, L, 64) → promedio con máscara → (B, 64) → Linear + ReLU → (B, H) → Linear → (B, 10). La última dimensión siempre es la cantidad de clases del problema: en el notebook eran 5 estrellas, acá 10 categorías. Lo que no cambia es el resto del recorrido.

Pregunta 2
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Emparejá cada decisión o uso con el conjunto que corresponde.

Reportar la estimación final de cómo se comporta el modelo con datos nuevos. — Prueba
Elegir cuántas épocas correr y qué tasa de aprendizaje usar. — Validación
Ajustar los pesos con descenso de gradiente. — Entrenamiento

Bien: entrenamiento ajusta, validación decide, prueba reporta una sola vez.

Resolución. Entrenamiento (60–80 %): ajusta pesos. **Validación** (10–20 %): afinar hiperparámetros, decidir cuándo frenar, comparar arquitecturas. **Prueba** (10–20 %): una evaluación imparcial al final. Si las decisiones se tomaran mirando prueba, dejaría de ser «nunca visto».

La respuesta correcta es: Reportar la estimación final de cómo se comporta el modelo con datos nuevos. → Prueba, Elegir cuántas épocas correr y qué tasa de aprendizaje usar. → Validación, Ajustar los pesos con descenso de gradiente. → Entrenamiento

Pregunta 3
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Las reseñas tienen en promedio unas 32 palabras y el tensor se rellena hasta `L = 60` (sin contar el efecto del truncado). ¿Aproximadamente qué porcentaje de las posiciones del tensor es relleno? Respondé un entero.

Respuesta: 47

(60 − 32) / 60 ≈ 47 %. En el notebook, medido sobre el tensor real, «más de la mitad»: las reseñas truncadas a 60 y la mediana de 26 empujan el número un poco más arriba.

Resolución. Relleno ≈ (60 − 32) / 60 ≈ **47 %**. Medido sobre el tensor real el notebook da algo más de la mitad, porque la distribución es asimétrica (mediana 26). Ese es el precio de trabajar con secuencias de largo fijo, y lo que el bucketing recupera.

La respuesta correcta es: 47

Pregunta 4

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Se entrena sobre **180.000** reseñas con mini-lotes de **64**. ¿Cuántas actualizaciones de pesos hay en una época? Respondé un entero (redondeá hacia arriba si el último lote queda incompleto).

Respuesta: 2813

180.000 / 64 = 2.812,5 → 2.813 lotes, una actualización por lote. Con full-batch sería 1 por época; con incremental, 180.000.

Resolución. 180.000 / 64 = 2.812,5 → **2.813** actualizaciones por época. Full-batch: 1 (estable pero un solo paso); incremental: 180.000 (avanza mucho, ruidoso y lento en tiempo real); mini-batch: dos órdenes de magnitud menos que el incremental y aprovecha el paralelismo del hardware.

La respuesta correcta es: 2813

Pregunta 5

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Emparejá cada esquema de actualización con su descripción.

Una sola actualización por época, calculada sobre el conjunto entero: camino estable pero costoso, y a veces el conjunto no entra en memoria. → Full-batch

Una actualización por ejemplo: la red se mueve muchísimo, pero cada paso persigue el error de un único caso y el recorrido es errático. → Incremental / SGD

El gradiente se calcula sobre 32, 64 o 128 ejemplos: menos ruidoso que de a uno, eficiente, y aprovecha el paralelismo de la GPU. → Mini-batch

Bien: full-batch estable pero caro, incremental rápido pero ruidoso, mini-batch el equilibrio de la práctica.

Resolución. Full-batch: todo el conjunto, una actualización por época. **Incremental / SGD**: un ejemplo por actualización, con los datos barajados para que las actualizaciones no se cancelen. **Mini-batch**: 32–128 ejemplos el equilibrio entre ruido y eficiencia. En el notebook, full-batch casi no se movió con un solo paso, incremental hizo 1.500 pasos lentísimos, y mini-batch avanzó bien con dos órdenes de magnitud menos de pasos.

La respuesta correcta es: Una sola actualización por época, calculada sobre el conjunto entero: camino estable pero costoso, y a veces el conjunto no entra en memoria. → Full-batch, Una actualización por ejemplo: la red se mueve muchísimo, pero cada paso persigue el error de un único caso y el recorrido es errático. → Incremental / SGD, El gradiente se calcula sobre 32, 64 o 128 ejemplos: menos ruidoso que de a uno, eficiente, y aprovecha el paralelismo de la GPU. → Mini-batch

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Un filtro de spam procesó 13 correos: 8 eran spam y 5 legítimos. De los 8 spam, marcó 5 como spam y dejó pasar 3. De los 5 legítimos, marcó 2 como spam. Con spam como clase positiva, respondé en **porcentaje con un decimal**: accuracy = 61.5 %, precisión = 71.4 %, recall = 62.5 %.

Resolución. TP = 5, FN = 3, FP = 2, TN = 3. **Accuracy** = (5 + 3)/13 = 61,5 %. **Precisión** = 5/(5 + 2) = 71,4 %: de lo marcado como spam, cuánto lo era. **Recall** = 5/(5 + 3) = 62,5 %: de todo el spam, cuánto se atrapó. Precisión baja = correos buenos a la basura; recall bajo = spam en la bandeja. Se oponen.

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Emparejá cada situación con lo que conviene hacer.

Error alto incluso sobre los datos de entrenamiento. → Más datos, o una red más profunda o más ancha

Hay pocos datos etiquetados y no se pueden conseguir más; el modelo subajusta. → Aumento de datos (data augmentation)

Error bajo en entrenamiento, alto en datos nuevos. → Regularización: L1/L2 o dropout

Bien: sobreajuste → regularizar; subajuste → más datos o más modelo; sin datos nuevos → aumentarlos.

Resolución. Sobreajuste (bien en entrenamiento, mal afuera) → regularización L1/L2, dropout. Subajuste (mal en todos lados) → más datos, red más profunda o más ancha, más features; y si no hay más datos, *data augmentation*. Cuidado con el péndulo: agrandar para curar el subajuste puede empujar al sobreajuste.

La respuesta correcta es: Error alto incluso sobre los datos de entrenamiento. → Más datos, o una red más profunda o más ancha, Hay pocos datos etiquetados y no se pueden conseguir más; el modelo subajusta. → Aumento de datos (data augmentation), Error bajo en entrenamiento, alto en datos nuevos. → Regularización: L1/L2 o dropout

Pregunta 8

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

En un conjunto donde el 99 % de los correos es legítimo, un filtro que dice «nunca es spam» tiene 99 % de accuracy. ¿Qué métrica lo delata?

Seleccione una:

a. La accuracy sobre el conjunto de prueba, que sería mucho más baja que sobre entrenamiento.

b. El recall de la clase spam: de todo el spam real no atrapó nada, así que vale 0. ✓ Sí. Precisión no se puede ni calcular (no marcó nada como spam: 0/0), y el recall es 0/todo el spam = 0. La accuracy alta era el desbalance, no el modelo.

c. La precisión de la clase spam: marcó como spam muchos correos legítimos.

d. La pérdida de entrenamiento: un modelo que siempre predice lo mismo no puede tener pérdida baja.

Resolución. Con clases desbalanceadas la accuracy se explica por la clase mayoritaria. Precisión y recall se calculan por clase: para «nunca es spam», recall(spam) = 0 y la precisión no está definida. Por eso el notebook mira precisión y recall por estrella, y la matriz de confusión.

La respuesta correcta es: El recall de la clase spam: de todo el spam real no atrapó nada, así que vale 0.

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Se agrega `nn.Dropout(0.3)` al clasificador. ¿Qué hace esa capa y por qué importa llamar a `modelo.eval()` antes de medir?

Seleccione una:

a. Elimina permanentemente el 30 % de las neuronas menos útiles al terminar cada época; `eval()` guarda esa poda.

b. Suma un término a la pérdida que penaliza los pesos grandes; `eval()` quita ese término para medir la pérdida real.

c. Apaga neuronas al azar tanto al entrenar como al evaluar; `eval()` solo fija la semilla para que el apagado sea reproducible.

d. Al entrenar apaga neuronas al azar para que ninguna domine; en `eval()` deja de apagar y usa la red completa. ✓ Sí. Es la analogía del equipo donde algunos jugadores se sientan al azar en los entrenamientos. Medir sin eval() sería evaluar con el equipo incompleto en cada pasada.

Resolución. Dropout apaga neuronas al azar en cada paso de entrenamiento, para que ninguna se vuelva dominante y el modelo generalice. En modo `eval()` no apaga ninguna. Por eso `modelo.eval()` antes de medir y `modelo.train()` antes de seguir entrenando; olvidarlo es un bug difícil de encontrar.

La respuesta correcta es: Al entrenar apaga neuronas al azar para que ninguna domine; en `eval()` deja de apagar y usa la red completa.

Pregunta 10

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Se guarda el clasificador entrenado con `torch.save(modelo.state_dict(), "clasificador.pt")`. ¿Qué hace falta para volver a usarlo en otra sesión?

Seleccione una:

a. Cargar el `state_dict` en cualquier `nn.Module` con la misma cantidad de parámetros; las formas se ajustan solas.

b. Cargar el archivo con `torch.load`: devuelve el modelo completo listo para predecir, con la arquitectura incluida.

c. Guardar además el `DataLoader` y el optimizador, porque sin ellos los pesos no se pueden interpretar.

d. Construir el modelo con la misma arquitectura, cargarle el `state_dict` con `load_state_dict`, y pasarlo a `eval()` antes de predecir.

Sí. El state_dict es un diccionario con los tensores de los parámetros; no incluye la clase. eval() importa en cuanto haya dropout o batch norm, y conviene acostumbrarse.

Resolución. Lo que se guarda no es el objeto sino su `state_dict`. Para recuperarlo: construir el modelo con la misma arquitectura, `load_state_dict`, y `eval()` (cambia el comportamiento de dropout y batch norm). En la Unidad 2 esto se convierte en el almacén de checkpoints con firma.

La respuesta correcta es: Construir el modelo con la misma arquitectura, cargarle el `state_dict` con `load_state_dict`, y pasarlo a `eval()` antes de predecir.

Pregunta 1

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Durante el entrenamiento, la pérdida sobre el conjunto de **entrenamiento** sigue bajando época tras época, mientras la de **validación** se aplana y después empieza a subir. ¿Qué está pasando y qué se hace?

Seleccione una:

a. Sobreajuste: el modelo memoriza los ejemplos vistos y deja de generalizar. Se regulariza (dropout, penalización L2) o se frena antes.

Sí. La separación entre las dos curvas es la firma. Y no es casualidad en este modelo: casi 1,5 millones de parámetros dedicados a memorizar palabras sueltas, más parámetros que ejemplos.

b. El conjunto de validación quedó mal partido y contiene ejemplos más difíciles; hay que rehacer el split.

c. La tasa de aprendizaje es demasiado grande y la validación diverge; hay que bajarla.

d. Subajuste: el modelo todavía no captura los patrones; hay que agrandarlo o entrenar más épocas.

Resolución. Entrenamiento baja, validación se aplana o sube: **sobreajuste**. El modelo anda bien sobre lo que vio y mal sobre lo nuevo. Remedios: regularización L1/L2 (penalizar pesos grandes), dropout (apagar neuronas al azar al entrenar), frenar antes. Subajuste es el otro modo de fallar: mal en todos lados; se ataca con más datos o más modelo.

La respuesta correcta es: Sobreajuste: el modelo memoriza los ejemplos vistos y deja de generalizar. Se regulariza (dropout, penalización L2) o se frena antes.

Pregunta 2
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un clasificador de cinco clases aplica `softmax` al final de su `forward`, y se entrena con `nn.CrossEntropyLoss`. ¿Qué pasa al entrenar?

Seleccione una:

a. El gradiente no llega a la capa de salida, porque la softmax no tiene derivada en la clase ganadora.
b. PyTorch lanza un error de forma, porque la pérdida espera logits y recibe una fila que suma 1.
c. Entrena y la pérdida baja, pero aprende mucho peor: la softmax se aplica dos veces y los logits quedan aplastados. ✓ La pérdida aplica su propia softmax sobre lo que recibe. Si ya recibe probabilidades (entre 0 y 1), la segunda softmax las convierte en una distribución casi uniforme y el gradiente se achica. Es un bug silencioso: por eso la convención es que el modelo devuelva logits.
d. La pérdida arranca en cero, porque las probabilidades ya están normalizadas y no hay nada que corregir.

Resolución. `CrossEntropyLoss` = `log_softmax` + tomar el valor de la clase correcta con signo cambiado. Si el modelo ya aplicó softmax, la pérdida aplica una segunda sobre números entre 0 y 1, cuya salida es casi uniforme: el modelo puede mejorar, pero muy poco y muy lento, y ningún error lo delata. Regla: el modelo devuelve **logits**; la softmax se aplica solo cuando querés leer probabilidades.

La respuesta correcta es: Entrena y la pérdida baja, pero aprende mucho peor: la softmax se aplica dos veces y los logits quedan aplastados.

Pregunta 3
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

Un parámetro `w` tiene, en cierto lote, derivada de la pérdida igual a **12** (la calcula autograd; no hace falta derivar nada). Si se llama `backward()` dos veces seguidas sobre ese mismo lote sin limpiar nada, `w.grad` marca [24] ✓. En el loop de entrenamiento, lo que evita que cada paso use la suma de toda la historia es [optimizador.zero_grad()] ✓, y se llama [antes de backward(), en cada iteración] ✓.

Resolución. `backward()` **suma** al `.grad` existente: dos llamadas dan 24. Es deliberado (permite acumular varios lotes cuando uno grande no entra en memoria), pero obliga a limpiar en cada iteración con `zero_grad()`, antes de `backward()`. Olvidarlo es el bug más frecuente del deep learning, y no da error.

Pregunta 4
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

El corpus de reseñas viene ordenado por etiqueta: primero las 40.000 de 1 estrella, después las de 2, y así. Si se toma el primer 90 % para entrenar y el último 10 % para validar, ¿qué pasa?

Seleccione una:

a. PyTorch detecta que las clases están desbalanceadas entre los conjuntos y lanza una advertencia al crear el DataLoader.
b. El modelo entrena bien pero la validación da accuracy muy baja, porque nunca vio reseñas de 5 estrellas.
c. La validación queda solo con reseñas de 5 estrellas; predecir siempre 5 daría 100 % de accuracy sin significar nada. ✓ Sí. Es un desastre silencioso: no hay error, solo un número engañoso. Por eso random_split baraja antes de partir, y por eso hay que mirar la estructura del conjunto antes de cortarlo.
d. No pasa nada grave: el 90 % de entrenamiento contiene las cinco clases y el modelo generaliza al 10 % restante.

Resolución. Partir por posición un conjunto ordenado por clase deja la validación con una sola clase. Un modelo que prediga siempre esa clase saca 100 %. `random_split` baraja antes de partir, y el `shuffle=True` del DataLoader de entrenamiento evita que cada época muestre las clases en bloque. Vale la pena chequear siempre este tipo de estructura.

La respuesta correcta es: La validación queda solo con reseñas de 5 estrellas; predecir siempre 5 daría 100 % de accuracy sin significar nada.

Pregunta 5
Correcta
Se puntúa 1,00 sobre 1,00
Marcar pregunta

¿Cuántos parámetros entrenables tiene `nn.Linear(2, 3)`, una capa con 2 entradas y 3 perceptrones?

Respuesta: [9] ✓

3 × 2 pesos (una fila por perceptrón, una columna por entrada) + 3 sesgos = 9.

Resolución. `nn.Linear(2, 3)` crea `weight` de forma (3, 2) y `bias` de forma (3,): **9** parámetros. Regla general: (entradas + 1) × salidas.

La respuesta correcta es: 9

Pregunta 6

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

En un ejemplo, el modelo le asigna probabilidad **0,9** a la clase correcta. ¿Cuánto vale la entropía cruzada de ese ejemplo? Respondé con tres decimales.

Respuesta: 0.105

−ln(0,9) = 0,105. Acertar con seguridad casi no cuesta.

Resolución. Para un ejemplo, la entropía cruzada es −ln(p_correcta): −ln(0,9) = **0,105**. Con p = 0,5 sería 0,69; con p = 0,1, 2,30; con p = 1/5 (modelo sin entrenar), ln 5 = 1,61. Cuanta más probabilidad a la correcta, menos pérdida, y equivocarse con seguridad cuesta carísimo.

La respuesta correcta es: 0,105

Pregunta 7

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Se entrena el mismo modelo con tres tasas de aprendizaje: 0,01, 0,5 y 50. Con la más grande, la pérdida **sube** en vez de bajar. ¿Por qué?

Seleccione una:

a. Porque una tasa alta hace que los gradientes se acumulen entre iteraciones y crezcan sin control.

b. Porque una tasa mayor que 1 invierte el signo de la actualización y el modelo sube por la pendiente.

c. Porque el paso es tan largo que se pasa del mínimo en cada iteración y termina más arriba de donde empezó: diverge.

Sí. La derivada dice hacia dónde bajar, pero un paso enorme salta al otro lado del valle y aterriza más alto. Con 0,01 baja pero casi no se mueve; con 0,5 baja rápido y se estabiliza.

d. Porque con tasa grande el gradiente se calcula sobre menos ejemplos y se vuelve más ruidoso.

Resolución. La tasa de aprendizaje escala el paso: chica → precisa pero lenta; razonable → baja rápido y se estabiliza; muy grande → se pasa del mínimo en cada paso y la pérdida sube. Es la decisión más sensible del entrenamiento, y por eso existen los optimizadores adaptativos.

La respuesta correcta es: Porque el paso es tan largo que se pasa del mínimo en cada iteración y termina más arriba de donde empezó: diverge.

Pregunta 8

Parcialmente correcta

Se puntúa 0,67 sobre 1,00

Marcar pregunta

Emparejá cada situación con la función de pérdida que corresponde. Hay una opción de más.

Predecir el precio de una casa; hay unas pocas casas con precios disparatados que se consideran **ruido** y no deberían dominar el ajuste. → Error absoluto medio (L1)

Predecir un tiempo de entrega; un error grande en un solo pedido es **mucho más grave** que varios errores chicos. → Error cuadrático medio (L2)

Clasificar reseñas en 5 clases; el modelo devuelve logits y las etiquetas son índices 0–4. → Softmax

Algún par no cierra. Releé la resolución: la clave está en si los errores grandes hay que *ignorarlos* (L1) o *evitarlos a toda costa* (L2).

Ha seleccionado correctamente 2.

Resolución. L1 promedia |error|: un caso atípico pesa en proporción, así que si los atípicos son ruido, L1 no se deja arrastrar. **L2** promedia error^2: un error grande pesa muchísimo, lo que empuja al modelo a evitar equivocarse mucho en pocos casos, exactamente lo que se quiere cuando un error grande es grave. **Entropía cruzada** es la pérdida de clasificación: compara la distribución predicha con la etiqueta (índice en PyTorch, one-hot en la teoría). **Softmax** no es una pérdida. En el notebook, con un atípico L1 creció 5 veces y L2 más de 100.

La respuesta correcta es: Predecir el precio de una casa; hay unas pocas casas con precios disparatados que se consideran **ruido** y no deberían dominar el ajuste. → Error absoluto medio (L1), Predecir un tiempo de entrega; un error grande en un solo pedido es **mucho más grave** que varios errores chicos. → Error cuadrático medio (L2), Clasificar reseñas en 5 clases; el modelo devuelve logits y las etiquetas son índices 0–4. → Entropía cruzada

Pregunta 9

Correcta

Se puntúa 1,00 sobre 1,00

Marcar pregunta

Las reseñas tienen en promedio unas 32 palabras y el tensor se rellena hasta `L = 60` (sin contar el efecto del truncado). ¿Aproximadamente qué porcentaje de las posiciones del tensor es relleno? Respondé un entero.

Respuesta: 47

(60 − 32) / 60 ≈ 47 %. En el notebook, medido sobre el tensor real, «más de la mitad»: las reseñas truncadas a 60 y la mediana de 26 empujan el número un poco más arriba.

Resolución. Relleno ≈ (60 − 32) / 60 ≈ **47 %**. Medido sobre el tensor real el notebook da algo más de la mitad, porque la distribución es asimétrica (mediana 26). Ese es el precio de trabajar con secuencias de largo fijo, y lo que el bucketing recupera.

La respuesta correcta es: 47